Musanze Harvest Intelligence Service

Technical report - SWE 3513 Individual Assignment 2

Item	Value
Full name	YOUR FULL NAME
Registration number	25/27821
Institutional username	ug2527821@ines.ac.rw
Verification code	**AI-A2-7821**
Seed (LLLL)	7821
Repository / commit	PASTE_REPO_URL / PASTE_COMMIT_HASH

1. Identity and decision problem

Stakeholder. The field officer of a potato cooperative in Musanze, who receives farm records in inconsistent spreadsheets and needs a local, offline-capable tool to support planning.

Prediction questions. (1) How many tons of potatoes will a farm harvest? (regression) (2) Is a farm at high production risk, meaning its yield per hectare is below the median? (classification) (3) Which farms share similar growing conditions? (clustering)

Ethical boundary. This is a classroom decision-support prototype trained on synthetic data. It does not replace an agronomist and must never be used to decide access to finance, insurance or public services. Every prediction needs human review before any action is taken.

Work done. One command (run_all.py) regenerates the dataset from the personal seed, validates and cleans it, trains and evaluates three kinds of model, and writes all artifacts. predict.py and a FastAPI service serve the saved models, and a pytest suite checks success and failure paths.

2. Data and reproducibility

Personal seed rule. The seed is LLLL = 7821, used with numpy.random.default_rng(7821). Base rows = 240 + (7821 mod 41) = **271**. Then m = round(0.05 x 271) = 14 values were set to missing in each of soil_moisture_pct and fertilizer_kg, and three exact duplicate rows were appended, giving **274** raw rows.

Column	Generation rule	Use
rainfall_mm	Uniform 600-1400	Numeric feature
avg_temp_c	Uniform 12-26	Numeric feature
soil_moisture_pct	Uniform 20-80	Numeric feature
fertilizer_kg	Uniform 0-250	Numeric feature
field_size_ha	Uniform 0.2-4.0	Numeric feature
season	Random A or B	Categorical feature
cooperative_support	Random 0 or 1	Binary feature
yield_tons	Given formula + N(0, 0.80) noise, clipped at 0.50	Regression target
high_risk	Yield per hectare below its median	Classification target

Data-quality evidence

Check	Result
Raw rows	274
Missing values	{"soil_moisture_pct": 15, "fertilizer_kg": 14}
Exact duplicate rows	3
Rows after de-duplication	271
Range, category, schema checks	Passed (validation.py)

The missing values are not filled before the split. They are imputed inside the scikit-learn pipeline, using statistics learned from the training rows only.

Dataset fingerprint

SHA-256 of the final CSV bytes: `5f9cfffa323e458759ae5c0567fd1cf75e1a364dace30dd7541d2e4fa8107eda`

CSV files are written with four decimals and Unix line endings on every operating system, so the hash is identical on Windows, Linux and macOS. Re-running run_all.py reproduces the same hash and the same metrics.

3. Pipeline design

Train/test separation. After de-duplication (271 rows), one fixed split is made with train_test_split(test_size=0.20, random_state=7821, stratify=high_risk): 216 training and 55 test rows. All three tasks use this split, and the test set is used only for final reporting.

Preprocessing flow (one ColumnTransformer, reused at training and inference):

Columns	Steps
rainfall, temperature, soil moisture, fertilizer, field size	Median imputation, then standard scaling
season	Most-frequent imputation, then one-hot encoding (unknown values ignored)
cooperative_support	Most-frequent imputation (already 0/1)

Leakage controls. yield_tons, yield per hectare and high_risk are never used as classification features; assert_no_leakage() stops the run if they appear. Clustering uses features only. Imputers, encoders and scalers are fitted inside a Pipeline on training data, including inside each cross-validation fold.

Model candidates

Task	Baseline	Candidates	Selection rule
Regression	Mean predictor	Linear regression, random forest	Lowest 5-fold CV RMSE on training data
Classification	Majority class	Logistic regression, random forest	Highest 5-fold CV F1 on training data
Clustering	-	K-means, K = 2..6	Highest silhouette score

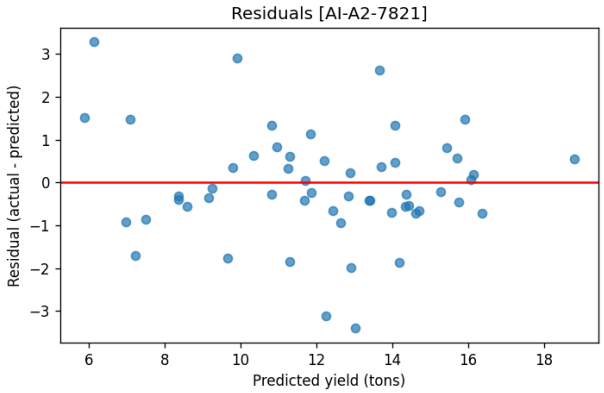
All random states are set to the seed. Final pipelines are saved with joblib in models/ and loaded by both predict.py and app.py, so training and inference apply identical transformations.

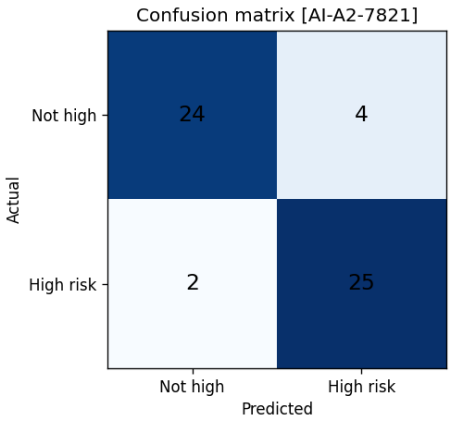
4. Regression and classification

Regression	CV RMSE (train)	MAE	RMSE	R2
mean_baseline	2.6947	2.4717	3.0551	-0.0
linear_regression (selected)	1.0262	0.933	1.2539	0.8315
random_forest	1.2776	1.3052	1.5931	0.7281

Classifier	CV F1	Precision	Recall	F1	ROC-AUC	[[TN,FP],[FN,TP]]
majority_baseline	0.2625	0.0	0.0	0.0	0.5	[[28, 0], [27, 0]]
logistic_regression (selected)	0.9526	0.8621	0.9259	0.8929	0.9841	[[24, 4], [2, 25]]
random_forest	0.9312	0.9259	0.9259	0.9259	0.9881	[[26, 2], [2, 25]]

Selected. Regression: linear_regression. Classification: logistic_regression. linear_regression has the lowest 5-fold CV RMSE on the training set (1.0262); test set used only for reporting. On the test set random_forest scored a higher F1 (0.9259) than logistic_regression (0.8929), but the choice was fixed by cross-validation on training data, so the test set stays an unbiased check; with only 55 test rows this difference is a few farms.





Interpretation

Residuals. The mean error is +0.08 t and 71 % of test farms are within 1 t, so errors are centred and the model is not systematically biased. The linear model beats the mean baseline (R2 -0.0) clearly, which fits the data: the yield formula is almost linear, so the more flexible random forest has no advantage.

Confusion matrix. [[24, 4], [2, 25]] (rows = actual, columns = predicted). The majority baseline never predicts high risk (recall 0), so accuracy alone would have hidden its uselessness.

Regression error (my interpretation). Row 116 has the largest test error: actual 9.62 t, predicted 13.03 t (error +3.40 t). Its conditions were rainfall 709 mm, temperature 25.2 C, soil moisture nan %, fertilizer 186 kg, field 3.03 ha, season A. The data contain random noise with a standard deviation of 0.80 t, so an error of about 3 t (roughly four noise standard deviations) shows a large unpredictable component that no model can learn, not a data-leak or coding problem.

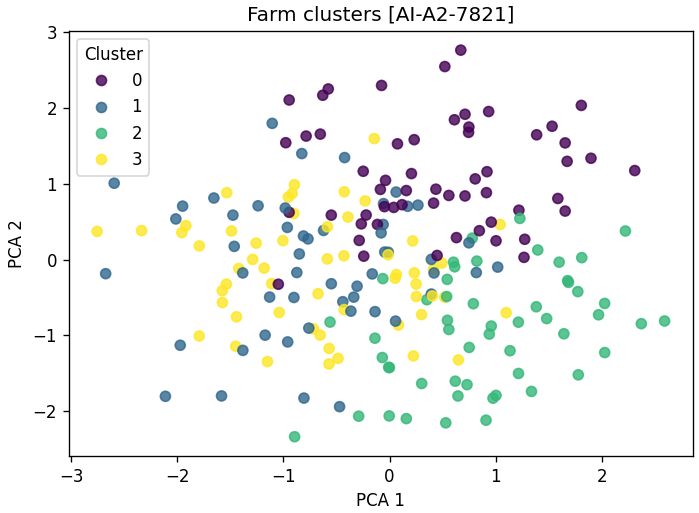
Classification error (my interpretation). Row 3 is the most confident mistake: the farm is truly high risk but the model gave a high-risk probability of 0.21. Its yield per hectare is 5.35 t/ha against a median of 5.89 t/ha, so it sits close to the median cut-off that defines the label. Farms near that boundary are inherently hard to separate because the label is a hard threshold on a noisy quantity.

5. Clustering

Features were encoded and scaled with the training-only preprocessor. Targets were not used. K-means was run for K = 2 to 6 and K was chosen by silhouette score.

K	2	3	4	5	6
Silhouette	0.1336	0.1297	0.1414	0.1411	0.1394

Selected K = 4. K=4 has the highest silhouette score (0.1414). The scores are all low and close together, so the farms form overlapping groups rather than sharply separated ones; the clusters are a convenient summary, not natural categories.



Cluster profiles

Cluster	Farms	Profile (original units)	Season A	Support	Mean yield (post-hoc)
0	56	56 farms with high rainfall (1212.2), low temperature (17.0), high fertilizer (176.1).	52 %	54 %	13.87 t
1	52	52 farms with low rainfall (834.7), low temperature (16.8), low soil moisture (33.9).	56 %	44 %	9.854 t
2	52	52 farms with low fertilizer (45.8), high field size (2.9).	35 %	48 %	11.958 t
3	56	56 farms with low rainfall (879.7), high temperature (22.4), high soil moisture (63.8).	41 %	41 %	12.344 t

Farm cluster (my interpretation). Cluster 0: 56 farms with high rainfall (1212.2), low temperature (17.0), high fertilizer (176.1). Mean post-hoc yield is 13.87 t, and 54 % of its farms receive cooperative support. A field officer can treat these farms as one group when planning visits, inputs and advice, then check whether the group really behaves alike on the ground.

Operational meaning. Clusters let the officer group farms for visits, input delivery and advice. Yield per cluster is shown only to describe the groups after fitting; it was not used to build them.

6. API and testing

Endpoint	Purpose	Behaviour
GET /health	Liveness check	200 with status and whether models exist
GET /model-info	Model details	200 with code AI-A2-7821, selected models, cluster profiles; 503 if not trained
POST /predict	One farm record	200 with yield, risk, probability, cluster, code; 422 if invalid; 503 if not trained

Valid request and response

```
{"rainfall_mm":1000,"avg_temp_c":18,"soil_moisture_pct":55,"fertilizer_kg":120,
 "field_size_ha":1.5,"season":"A","cooperative_support":1}
```

Response: yield_estimate_tons, high_risk_prediction, high_risk_probability, cluster, cluster_profile, verification_code (AI-A2-7821) and a disclaimer.

Invalid request and response

```
{"rainfall_mm": 5000, ...}   ->   HTTP 422, detail[0].loc = ["body","rainfall_mm"],
                                  msg = "Input should be less than or equal to 1400"
```

A missing field, a wrong type (e.g. text for a number), an unknown field, season outside A/B and cooperative_support outside 0/1 are all rejected the same way. predict.py performs equivalent checks and prints a list of errors (exit code 1).

Test summary

Test	What it covers
test_valid_inference	Saved models load and return a sensible prediction with the verification code
test_missing_field / wrong_type / range_error	Failure paths of record validation
test_reproducibility	Same seed gives identical CSV bytes and row count
test_data_problems_present	Missing values and duplicates are really injected
test_seed_must_match_registration	A seed that is not LLLL is rejected
test_leakage_guard	Target-derived columns cannot be features
test_artifacts_exist	All required artifacts are written
test_api_responses	/health, /model-info, valid and invalid /predict (422)

Result: 10 passed, 1 warning in 12.43s

Failure behaviour. Bad input never reaches the model: it returns a clear 4xx error (API) or an error list (CLI). If models are missing the service answers 503 with the command needed to train them.

7. Model card and reflection

Section	Statement
Intended use	Classroom prototype to help a cooperative field officer estimate harvest, flag possible high-risk farms and group similar farms for planning.
Data status	Entirely synthetic, generated from a known formula and my personal seed. It contains no real farm records.
Limitations	Small data (271 rows, 55 test rows); a near-linear formula favours linear models; risk label is a median threshold, so about half of all farms are 'high risk' by construction; clusters are weakly separated; no real soil, pest, variety or market factors.
Risks	Over-trusting a number, applying it to real farms or regions it has never seen, and unfair treatment if the output were used to rank farmers.
Prohibited use	Deciding finance, insurance or public-service access; replacing an agronomist; use on real farms without re-training and validation on real data.
Human oversight	A qualified person reviews every prediction before action and can override it. Responses carry a disclaimer and the verification code.

Reflection

With real data I would first check how the records were collected and whether farmers differ systematically, then add missing real drivers (soil tests, seed variety, planting date, pests), replace the median-based risk label by a locally agreed risk definition, use grouped or time-based validation, report uncertainty intervals, and monitor the model for drift with agronomists involved. I would also compare the weakly separated clusters with local knowledge before using them operationally.